

Laboratorio di Programmazione

Lezione 7: Rune e Stringhe

1	Qual è l'output?	2
2	Carte	3
3	Qual è l'output?	4
4	Qual è l'output?	5
5	Qual è l'output?	6
6	Qual è l'output?	7
7	Conteggio	8
8	Trasformazione maiuscole/minuscole	9
9	Generazione sottostringhe	10
10	Spaziatura caratteri	11
11	Conteggio (2)	12
12	Rombo	13
13	Stringa alternata	14
14	Stringa palindroma	15
15	Triangolo	16
16	Ripetizione	17
17	Rotazioni	18

1 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     var y int = 'a'
6     for x := 1; x < 10; x++ {
7         fmt.Print(x+y, " ")
8     }
9     fmt.Println()
10 }
```

2 Carte

Sapendo che al codice Unicode 127153 (associato alla rappresentazione in bit Unicode/UTF-8 '\U0001F0B1') corrisponde il simbolo "asso di cuori", e che i codici successivi corrispondono alle carte successive (2 di cuori, 3 di cuori, ...), scrivere un programma che stampi tutte le carte da gioco dall'asso di cuori al 10 di cuori.

```
$ go run carte.go
Simbolo: 🂠 - Codice numerico in base 10: 127153
Simbolo: 🂡 - Codice numerico in base 10: 127154
Simbolo: 🂢 - Codice numerico in base 10: 127155
Simbolo: 🂣 - Codice numerico in base 10: 127156
Simbolo: 🂤 - Codice numerico in base 10: 127157
Simbolo: 🂥 - Codice numerico in base 10: 127158
Simbolo: 🂦 - Codice numerico in base 10: 127159
Simbolo: 🂧 - Codice numerico in base 10: 127160
Simbolo: 🂨 - Codice numerico in base 10: 127161
Simbolo: 🂩 - Codice numerico in base 10: 127162
```

3 Qual è l'output?

Analizzate l'output del seguente programma.

```
1 package main
2 import "fmt"
3
4 func main() {
5
6     s := "Ciao René!"
7     count := 0
8     for i, c := range s {
9         fmt.Print("In posizione ", i,
10             " inizia la sequenza di byte che codifica il carattere ",
11             string(c), "\n")
12         count++
13     }
14     fmt.Println("Lunghezza stringa in byte:", len(s))
15     fmt.Println("Lunghezza stringa in caratteri:", count)
16 }
```

4 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     s := "ciao, come va?"
6     /* s formata soltanto da caratteri US-ASCII */
7
8     fmt.Println(string(s[0]) + string(s[len(s)-2]) + string(s[len(s)-1]))
9 }
```

5 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     // s formata soltanto da caratteri US-ASCII
6     s := "Ciao, come va?"
7     for i := 0; i<len(s); i++ {
8         fmt.Print(string(s[i]))
9     }
10    fmt.Println()
11
12    // s formata anche da caratteri non US-ASCII
13    s = "Ciao, com'è?"
14    for i := 0; i<len(s); i++ {
15        fmt.Print(string(s[i]))
16    }
17    fmt.Println()
18 }
```

Ricordate:

- `s[i]` è il byte in posizione `i` di `s`, e in generale `s[i]` non codifica un carattere.
- per iterare sui *caratteri* della stringa si usa `for range`.

6 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     s := "ciao, come va?"
6     /* s formata soltanto da caratteri US-ASCII */
7     fmt.Println(s[6:10] + string(s[len(s)-1]))
8     fmt.Println(s[:5] + s[len(s)-4:])
9 }
```

7 Conteggio

Scrivere un programma che legga da standard input una stringa senza spazi e, considerando solamente l'insieme delle lettere dell'alfabeto inglese, stampi:

- il numero di lettere maiuscole;
- il numero di lettere minuscole;
- il numero di consonanti;
- il numero di vocali.

A tal fine definire e usare le seguenti funzioni:

- `èLetteraValida(l rune) bool`,
- `èMaiuscola(l rune) bool`,
- `èVocale(l rune) bool`.

Suggerimento: considerate il valore numerico del carattere e guardate la tabella ASCII.

Esempio d'esecuzione:

```
$ go run analisi.go
```

```
Ciaoà
```

```
Maiuscole: 1
```

```
Minuscole: 3
```

```
Vocali: 3
```

```
Consonanti: 1
```

```
$ go run analisi.go
```

```
Certo!Sto,bene!iiiiii
```

```
Maiuscole: 2
```

```
Minuscole: 10
```

```
Vocali: 5
```

```
Consonanti: 7
```

```
$ go run analisi.go
```

```
aaAA
```

```
Maiuscole: 2
```

```
Minuscole: 2
```

```
Vocali: 4
```

```
Consonanti: 0
```


8 Trasformazione maiuscole/minuscole

Scrivere un programma che legga da standard input una stringa, e la ristampi in maiuscolo e in minuscolo. A tal fine definire due funzioni:

- `inMaiuscolo(carattere rune) rune`
- `inMinuscolo(carattere rune) rune`

Esempio d'esecuzione:

```
$ go run trasforma.go
TestoDiProva!!!
Testo maiuscolo: TESTODIPROVA!!!
Testo minuscolo: testodiprova!!!

$ go run trasforma.go
Testo_Di_PrOvA....
Testo maiuscolo: TESTO_DI_PROVA....
Testo minuscolo: testo_di_prova....
```

9 Generazione sottostringhe

Scrivere un programma che:

1. legga da standard input una stringa senza spazi e formata solo da lettere dell'alfabeto inglese;
2. stampi le stringhe ottenute eliminando ripetutamente la prima e l'ultima lettera.

Esempio d'esecuzione:

```
$ go run sottostringhe.go
Parola in input: Prova
Sottostringa 1: Prova
Sottostringa 2: rov
Sottostringa 3: o
```

```
$ go run sottostringhe.go
Parola in input: Faro
Sottostringa 1: Faro
Sottostringa 2: ar
```

10 Spaziatura caratteri

Scrivere un programma che:

1. legga da standard input una stringa senza spazi e formata solamente da lettere dell'alfabeto inglese;
2. stampi la stessa stringa in modo tale che ogni lettera sia separata da quella successiva da uno spazio.

Esempio d'esecuzione:

```
$ go run spazia.go
```

```
Inserisci una stringa di testo: CiaoMondo!
```

```
C i a o M o n d o !
```

11 Conteggio (2)

Risolvete l'esercizio 7, ma usando le funzioni `unicode.IsLetter()` e `unicode.IsUpper()`. Prima di usarle leggete la documentazione!

Esempio d'esecuzione:

```
$ go run analisi.go
Ciao
Vocali maiuscole: 0
Consonanti maiuscole: 1
Vocali minuscole: 3
Consonanti minuscole: 0
```

```
$ go run analisi.go
Certo!Sto,bene
Vocali maiuscole: 0
Consonanti maiuscole: 2
Vocali minuscole: 5
Consonanti minuscole: 5
```

```
$ go run analisi.go
aaAA
Vocali maiuscole: 2
Consonanti maiuscole: 0
Vocali minuscole: 2
Consonanti minuscole: 0
```

12 Rombo

Scrivere un programma che legga da standard input un intero n e stampi a video un rombo di asterischi con diagonali di lunghezza $2n+1$. A tale scopo definite due stringhe, `stringaSpazi` e `stringaAsterischi`, di lunghezza opportuna e formate solo da asterischi e spazi. Poi utilizzate le loro sottostringhe durante la stampa. Se n non è strettamente positivo stampate un messaggio d'errore.

Esempio d'esecuzione:

```
$ go run rombo.go
```

```
3
```

```
  *
 ***
*****
*****
*****
 ***
  *
```

```
$ go run rombo.go
```

```
0
```

```
Dimensione non sufficiente
```

13 Stringa alternata

Scrivere un programma che legge da standard input due stringhe US-ASCII senza spazi, s_1 ed s_2 , e stampa a video la stringa ottenuta alternando i loro caratteri (se una stringa finisce prima dell'altra, usate il trattino (-) come riempimento). A tal fine definite una funzione:

```
Alternata(s1, s2 string) (s string)
```

Esempio d'esecuzione

```
$ go run stringa_alternata.go
AC
BDF
ABCD-F
```

```
$ go run stringa_alternata.go
ciaone
mondo
cmioanodhoe-
```

```
$ go run stringa_alternata.go
esame
go
egsoa-m-e-
```

14 Stringa palindroma

Una stringa $s_1s_2 \dots s_n$ è *palindroma* se è uguale alla sua inversa, $s_ns_{n-1} \dots s_1$. Scrivete un programma che legge da standard input una stringa senza spazi e decide se è palindroma. A tal scopo scrivete una funzione `ÈPalindroma(s string) bool`.

Esempio d'esecuzione:

```
$ go run palindroma.go
anna
Palindroma
```

```
$ go run palindroma.go
anni
Non palindroma
```

```
$ go run palindroma.go
osso
Palindroma
```

```
$ go run palindroma.go
èssè
Palindroma
```

```
$ go run palindroma.go
èsse
Non palindroma
```

15 Triangolo

Scrivete un programma che legge da standard input un intero n e, usando un solo ciclo `for` ed una sola variabile `string`, stampa un triangolo rettangolo di asterischi di base e altezza n (stampare un errore se $n \leq 0$).

Esempio d'esecuzione:

```
$ go run triangolo.go
-2
Dimensione non sufficiente
```

```
$ go run triangolo.go
5
*
**
***
****
*****
```


16 Ripetizione

Scrivere un programma che legge da standard input un intero n ed una stringa senza spazi, e stampa n volte la stringa letta, intervallando le occorrenze della stringa con il trattino -.

Esempio d'esecuzione:

```
$ go run ripetizione.go
5 test
test-test-test-test-test
```

17 Rotazioni

Scrivere un programma che legge da standard input una stringa senza spazi e, usando *un solo ciclo*, stampa tutte le sue possibili *rotazioni*.

Esempio d'esecuzione:

```
$ go run substr.go  
celoria  
Rotazioni:  
celoria  
eloriac  
loriace  
oriacel  
riacelo  
iacelor  
acelori
```